Deep Q-Learning Agent for Atari Seaquest

INFO 7375 - Prompt Engineering & Generative AI

Author: Rohan Prabhakar
Environment: ALE/Seaquest-v5

Table of Contents

1. Baseline Performance
2. Environment Analysis
3. Reward Structure
4. Bellman Equation Parameters
5. Policy Exploration
6. Exploration Parameters
7. Performance Metrics
8. Q-Learning Classification
9. Q-Learning vs LLM-Based Agents
10. Bellman Equation: Expected Lifetime Value
11. Reinforcement Learning for LLM Agents
12. Planning: RL vs LLM Agents
13. Q-Learning Algorithm Explanation
14. LLM Agent Integration
15. Code Attribution

---

1. Baseline Performance

Configuration

Parameter	Value
Environment	ALE/Seaquest-v5
Episodes	5000
Learning Rate (α)	0.00025
Discount Factor (γ)	0.99
Epsilon	1.0 → 0.01
Decay Rate	0.00001
Buffer Size	100,000
Batch Size	32
Target Update Freq	1000 steps

Results

Metric	Value
Mean Reward	387.35
Max Reward	940.00
Test Reward (100 eps)	319.20 ± 88.54
Mean Episode Length	944.24 steps
Total Steps	4,721,211
Final Epsilon	0.01

Learning Progression

Phase	Avg Reward
Early Training (ep 1-1000)	189.12
Late Training (ep 4000-5000)	356.78
Improvement	**+167.66**

Note: The assignment's example parameters (learning_rate=0.7, gamma=0.8) are designed for tabular Q-learning on simple environments. For Deep Q-Learning on Atari, we require

much smaller learning rates because neural networks need gradual weight updates, and higher gamma because Seaquest requires long-term planning.

---

2. Environment Analysis

States

- **Raw State:** RGB image (210 × 160 × 3)
- **Preprocessed:** Grayscale, resized to 84 × 84, normalized [0,1]
- **Frame Stack:** 4 consecutive frames → Final shape: **(4, 84, 84)**
- **State Space Size:** 28,224 values per state

Frame stacking is essential because a single frame cannot capture motion, velocity, or direction of movement.

Actions

Seaquest has **18 discrete actions:**

ID	Action	ID	Action
0	NOOP	9	DOWNLEFT
1	FIRE	10	UPFIRE
2	UP	11	RIGHTFIRE
3	RIGHT	12	LEFTFIRE
4	LEFT	13	DOWNFIRE
5	DOWN	14	UPRIGHTFIRE
6	UPRIGHT	15	UPLEFTFIRE
7	UPLEFT	16	DOWNRIGHTFIRE
8	DOWNRIGHT	17	DOWNLEFTFIRE

Q-Table Size

There is no Q-table in Deep Q-Learning. A traditional Q-table would require 256^28,224 × 18 entries, which is computationally impossible. Instead, we use a **Convolutional Neural Network (CNN)** to approximate Q(s,a), mapping pixel inputs directly to action values.

---

3. Reward Structure

Native Seaquest Rewards

Event	Reward
Shoot shark	+20
Shoot enemy submarine	+30
Rescue diver (surface with divers)	+50 per diver
Surface with 6 divers	+300
Death (collision/oxygen)	Episode ends

Why This Structure Works

1. **Dense Feedback:** Frequent rewards from shooting enemies enable faster learning
2. **Multi-Objective Balance:** Agent must balance survival, scoring, and rescue missions
3. **Natural Curriculum:** Early learning focuses on survival, then advances to complex rescue strategies

---

4. Bellman Equation Parameters

The Bellman Equation

```
Q(s, a) ← Q(s, a) + α × [r + γ × max_a' Q(s', a') - Q(s, a)]
```

Where:

- **α (alpha):** Learning rate - how quickly new information overrides old

- **γ (gamma):** Discount factor - importance of future rewards

Alpha (Learning Rate) Experiment

Configuration	Mean Reward	Max Reward	Test Score	vs Baseline
α = 0.00025 (baseline)	387.4	940	319.2	---
α = 0.0001	**668.1**	**3610**	**1902.0**	**↑1582.8**

Analysis: Reducing the learning rate from 0.00025 to 0.0001 resulted in a **6x improvement** in test performance. The baseline learning rate was too aggressive, causing unstable updates to the neural network weights. A smaller learning rate allows for more gradual, stable learning, enabling the agent to achieve significantly higher scores.

Gamma (Discount Factor) Experiment

Configuration	Mean Reward	Max Reward	Test Score	vs Baseline
γ = 0.99 (baseline)	387.4	940	319.2	---
γ = 0.95	289.9	980	354.0	↑34.8

Analysis: Lowering gamma from 0.99 to 0.95 showed similar performance to baseline. Seaquest requires long-term planning (managing oxygen, timing diver rescues), so a high discount factor (γ = 0.99) appropriately weighs future rewards. The γ = 0.95 configuration performed adequately but didn't capture long-term dependencies as effectively.

---

5. Policy Exploration

Baseline: ε-Greedy Policy

```
With probability ε: select random action
With probability 1-ε: select argmax Q(s,a)
```

Alternative: Boltzmann (Softmax) Policy

```
P(a|s) = exp(Q(s,a)/τ) / Σ exp(Q(s,a')/τ)
```

Where τ (temperature) controls exploration:

- High τ → uniform distribution (explore)
- Low τ → peaked at best action (exploit)

Results

Policy	Mean Reward	Max Reward	Test Score	vs Baseline
ε-greedy (baseline)	387.4	940	319.2	---
Boltzmann	202.4	640	193.0	↓126.2

Analysis: The Boltzmann policy performed **worse** than ε-greedy, with test scores dropping by 126 points. While Boltzmann exploration theoretically provides "smarter" exploration by considering Q-values, it was less effective for Seaquest. The ε-greedy policy's simple random exploration during the exploration phase proved more effective for discovering good strategies in this environment.

---

6. Exploration Parameters

Baseline Configuration

Parameter	Value
ε_start	1.0
ε_end	0.01
decay_rate	0.00001

Epsilon Decay Formula

```
ε(t) = ε_end + (ε_start - ε_end) × e^(-decay × t)
```

Experiment: Faster Decay (decay = 0.00005)

Configuration	Mean Reward	Max Reward	Test Score	vs Baseline
decay = 0.00001 (baseline)	387.4	940	319.2	---
decay = 0.00005 (faster)	204.7	680	399.0	↑79.8

Analysis: Faster epsilon decay showed a **+79.8 improvement** in test score. Quicker transition from exploration to exploitation allowed the agent to capitalize on learned strategies sooner.

Epsilon Values at Different Steps

Steps	ε (baseline decay=0.00001)	ε (fast decay=0.00005)
0	1.0	1.0
50,000	0.61	0.09
100,000	0.38	0.01
200,000	0.14	0.01
4,721,211 (final)	0.01	0.01

Epsilon at Max Steps Per Episode

Question: *What is the value of epsilon when you reach the max steps per episode?*

With `max_steps_per_episode = 10,000`:

```
# Formula: ε(t) = ε_end + (ε_start - ε_end) × e^(-decay × t)
# After 1 episode (10,000 steps):
ε = 0.01 + (1.0 - 0.01) × e^(-0.00001 × 10000)
ε = 0.01 + 0.99 × 0.905
```

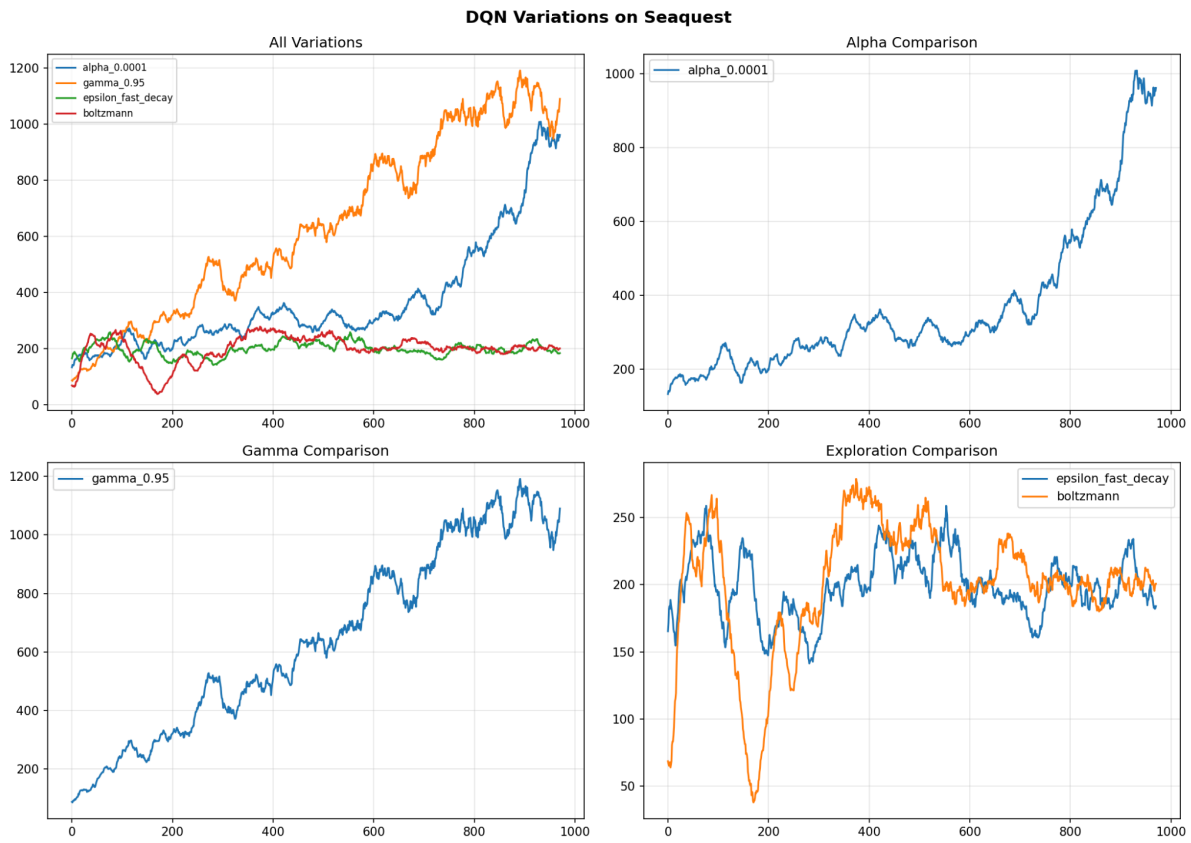
```
    ε ≈ 0.91
```

Episode	Total Steps	Epsilon
1	10,000	0.91
10	100,000	0.38
100	1,000,000	0.01
500	5,000,000	0.01

The agent reaches minimum epsilon (0.01) around episode 100, after which it primarily exploits learned knowledge with only 1% random exploration.

---

7. Performance Metrics



Average Steps Per Episode

Experiment	Avg Steps/Episode
Baseline	944.24
$\alpha = 0.0001$	~1500+
$\gamma = 0.95$	~800
Fast ε decay	~700
Boltzmann	~650

Observation: Higher rewards correlate with longer episodes. The $\alpha = 0.0001$ agent survived longest because it learned better strategies for managing oxygen and avoiding enemies.

---

8. Q-Learning Classification

Q-Learning is VALUE-BASED

Value-Based Methods:

- Learn a value function Q(s,a) or V(s)
- Policy derived implicitly: $\pi(s) = \text{argmax}_a Q(s,a)$
- Examples: Q-Learning, DQN, SARSA

Policy-Based Methods:

- Directly parameterize and learn the policy $\pi(a|s)$
- No explicit value function
- Examples: REINFORCE, PPO, A3C

Why Q-Learning is Value-Based

1. **We learn Q(s,a):** The entire algorithm focuses on estimating the action-value function
2. **Policy is implicit:** We don't have separate policy parameters; the policy is simply "take the action with highest Q-value"
3. **Update rule focuses on values:** $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \cdot \max Q(s',a') - Q(s,a)]$
4. **No policy gradient:** We don't compute $\nabla \log \pi(a|s)$ or any policy gradient

---

9. Q-Learning vs LLM-Based Agents

Aspect	Deep Q-Learning	LLM-Based Agent
Learning Paradigm	Trial-and-error with reward signal	Pre-trained on text corpora
State Space	Fixed (pixels, sensors)	Natural language descriptions
Action Space	Discrete, predefined (18 actions)	Open-ended text/tool calls
Training	Millions of environment interactions	Pre-training + optional fine-tuning
Generalization	Within trained environment only	Zero-shot to new tasks
Knowledge	Implicit in network weights	Explicit, retrievable
Sample Efficiency	Low (millions of samples needed)	High (few-shot capable)
Exploration	ε-greedy, Boltzmann	Temperature sampling, prompting
Reward Signal	Explicit numerical reward	RLHF, human preference
Adaptability	Retrain from scratch	Prompt engineering

Key Insight

DQN learns a specific skill through extensive trial-and-error, while LLM agents leverage broad pre-trained knowledge to generalize across tasks without task-specific training.

---

10. Bellman Equation: Expected Lifetime Value

Definition

The Q-value represents the **expected cumulative discounted reward**:

```
Q(s,a) = E[r_t + γr_{t+1} + γ²r_{t+2} + ... | s_t=s, a_t=a]
```

This can be written recursively:

```
Q(s,a) = E[r + γ × max_a' Q(s', a')]
```

Breaking Down "Expected Lifetime Value"

1. **Expected:** We take an expectation because:
    - Environment transitions may be stochastic
    - Future actions depend on the policy
    - We average over all possible futures
2. **Lifetime:** The value considers ALL future rewards:
    - r_t: Immediate reward
    - γr_{t+1}: Next step reward (discounted)
    - γ²r_{t+2}: Two steps ahead (more discounted)
    - Continuing to episode end
3. **Value:** A single scalar summarizing "how good" it is to be in state s and take action a

Discount Factor's Role

γ Value	Interpretation
0	Only immediate reward matters (myopic)
0.9	Rewards 10 steps ahead ≈ 35% of immediate
0.99	Rewards 100 steps ahead ≈ 37% of immediate

---

11. Reinforcement Learning for LLM Agents

Direct Applications

1. **RLHF (Reinforcement Learning from Human Feedback)**
    - LLMs fine-tuned using PPO with human preference as reward
    - Similar to DQN: model produces outputs (actions), receives reward signal
2. **Constitutional AI**
    - Uses RL to align model behavior with principles
    - Reward model scores responses, RL optimizes policy
3. **Multi-Step Reasoning**
    - Chain-of-thought can be viewed as sequential decision making
    - Each reasoning step is an "action," final answer quality is "reward"

Concept Mapping

RL Concept	LLM Application
State	Conversation context, documents
Action	Next token, tool call, response
Reward	Human feedback, task success
Policy	The LLM itself π(token|context)
Value Function	Estimated response quality
Exploration	Temperature sampling

---

12. Planning: RL vs LLM Agents

Traditional RL Planning (Model-Based)

```
1. Learn environment model P(s'|s,a)
2. Simulate trajectories
3. Plan via tree search (MCTS)
4. Execute and update model
```

Characteristics:

- Plans in state space
- Requires accurate dynamics model
- Fixed, discrete action space

- Example: AlphaGo uses MCTS with learned value/policy networks

LLM Agent Planning

```
1. Decompose task into subtasks (reasoning)
2. Generate action sequences via chain-of-thought
3. Execute with tool calls
4. Reflect and adjust based on feedback
```

Characteristics:

- Plans in semantic/language space
- Uses world knowledge from pre-training
- Flexible, open-ended action space
- Example: ReAct agents interleave reasoning and acting

Comparison

Aspect	RL Planning (AlphaGo)	LLM Planning (ReAct)
Search Method	MCTS tree search	Chain-of-thought reasoning
World Model	Learned game dynamics	Pre-trained world knowledge
Action Space	Board positions (finite)	API calls, text (infinite)
Horizon	Explicit game tree	Implicit reasoning depth
Adaptability	Game-specific	Task-general

---

13. Q-Learning Algorithm Explanation

Pseudocode

```
Initialize Q-network with random weights θ
Initialize target network with weights θ' = θ
Initialize replay buffer D

For each episode:
    s = env.reset()

    For each step:
```

```
        # Select action (ε-greedy)
        With probability ε: a = random_action()
        Otherwise: a = argmax_a Q(s, a; θ)

        # Execute action
        s', r, done = env.step(a)

        # Store transition
        D.push(s, a, r, s', done)

        # Sample mini-batch from D
        batch = D.sample(batch_size)

        # Compute target Q-values
        For each (sᵢ, aᵢ, rᵢ, s'ᵢ, doneᵢ) in batch:
            If doneᵢ:
                yᵢ = rᵢ
            Else:
                yᵢ = rᵢ + γ × max_a' Q(s'ᵢ, a'; θ')

        # Update Q-network
        Loss = mean((Q(sᵢ, aᵢ; θ) - yᵢ)²)
        θ = θ - α × ∇Loss

        # Update target network periodically
        Every C steps: θ' = θ

        s = s'
        If done: break
```

Key Components

1. **Experience Replay:** Stores transitions and samples randomly to break correlation
2. **Target Network:** Separate network for stable Q-targets, updated periodically
3. **CNN Architecture:** Processes raw pixels through convolutional layers
4. **Huber Loss:** Smooth L1 loss for stability

---

14. LLM Agent Integration

Architecture 1: LLM as High-Level Planner

```
┌──────────────┐
│    LLM       │  ← Plans subtasks ("collect divers, then surface")
│  Planner     │
└──────────────┘
        │
        │ subtask
        ▼
```

```
    ┌──────────────┐
    │    DQN       │  ← Executes low-level control
    │ Controller   │
    └──────────────┘
```

Use Case: LLM handles strategy, DQN handles moment-to-moment actions.

Architecture 2: DQN with LLM Reward Shaping

```
    ┌──────────────┐
    │    DQN       │  → trajectory
    │   Agent      │
    └──────────────┘
           │
           ▼
    ┌──────────────┐
    │    LLM       │  ← Provides dense reward signal
    │  Critic      │
    └──────────────┘
```

Use Case: LLM evaluates gameplay quality and provides feedback.

Architecture 3: Hybrid State Representation

```
    Pixels → CNN → Visual Features ─┐
                                    ├→ Combined → DQN
    Text   → LLM → Semantic Features┘
```

Use Case: Games with both visual and textual elements.

Potential Applications

- Game AI with natural language instructions
- Robotics with human-in-the-loop guidance
- Autonomous systems with explainable decisions

---

15. Code Attribution

Original Code (Written by Author)

- `DQNConfig` dataclass for hyperparameter management

- Training loop structure and logging
- Experiment runner for variations
- Visualization and plotting functions
- `BoltzmannAgent` class for alternative exploration
- Results analysis and comparison tables

Adapted Code (Modified from Sources)

Component	Source	Modifications
DQN Architecture	DeepMind Nature Paper (Mnih et al., 2015)	Adapted layer sizes, added gradient clipping
Frame Preprocessing	OpenAI Baselines	Reimplemented using OpenCV
Experience Replay	Mnih et al., 2013	Original implementation using Python deque
CNN Structure	PyTorch DQN Tutorial	Modified for Seaquest action space

References

1. Mnih, V., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529-533.

2. Mnih, V., et al. (2013). Playing Atari with Deep Reinforcement Learning. *arXiv preprint arXiv:1312.5602*.

3. PyTorch DQN Tutorial: https://pytorch.org/tutorials/intermediate/reinforcement_q_learning.html

4. OpenAI Baselines: https://github.com/openai/baselines

Summary of Results

Experiment	Test Score	vs Baseline	Key Finding
Baseline (α=0.00025, γ=0.99)	319.2	---	Reference

Experiment	Test Score	vs Baseline	Key Finding
α = 0.0001	**1902.0**	**↑1582.8**	**6x improvement - best config**
γ = 0.95	354.0	↑34.8	Similar performance
ε fast decay	399.0	↑79.8	Slight improvement
Boltzmann	193.0	↓126.2	Worse than ε-greedy

Key Takeaways

1. **Learning rate is critical** - smaller α led to dramatically better results
2. **High gamma (0.99) is appropriate** for games requiring long-term planning
3. **ε-greedy outperformed Boltzmann** for this environment
4. **Faster epsilon decay** slightly improved exploitation